

PROYECTO INTEGRADOR — IX CUATRIMESTRE

Plan de Asignación & Análisis de Rúbrica

Diagnóstico de Inconsistencias Rúbrica vs Producción, Inventario de Entregables Faltantes y Matriz de Asignación de Tareas para el Equipo de 5 Desarrolladores.

PROYECTO

QUANTIFY SYSTEM

INTEGRANTES DEL EQUIPO

5 Desarrolladores

ASIGNATURA

Extracción de Conocimiento en BD

FECHA DE EMISIÓN

Agosto 18, 2026

1. Diagnóstico de Inconsistencias

El proyecto **QUANTIFY** se encuentra desplegado y funcionando en **entorno real de producción** con arquitectura de persistencia híbrida (MySQL + MongoDB), integración de Gemini AI, dashboard interactivo en React, aplicación para smartwatch y conexión WebSockets. La guía del proyecto integrador (Guia_Proyecto_Integrador.md) presenta las siguientes fricciones técnicas que deben ser alineadas y justificadas ante la evaluación docente:

Tema / Etapa	Guía del Proyecto Integrador	Realidad en Producción (QUANTIFY) & Estrategia
5. Simulación de Datos	Exige generar un CSV sintético estático mediante script Python con <code>np.random.seed(2026)</code> .	Datos Vivos: En producción se ingieren registros reales de usuarios en MySQL y MongoDB. <i>Estrategia:</i> Crear el script de simulación sintética para fines académicos sin alterar las bases de datos de producción.
6. Data Warehouse	Requiere un esquema OLAP dedicado (Estrella / Copo de Nieve) con tablas de hechos y dimensiones.	Persistencia Híbrida OLTP: Se utiliza MySQL para ACID y MongoDB para logs masivos de eventos y hábitos. <i>Estrategia:</i> Modelar un Data Mart lógico en <code>/database/warehouse/</code> que documente la agregación analítica de la telemetría.
10, 11 y 13. Modelos & API	Asume modelos clasicos de ML en Python (Scikit-Learn) exportados en <code>.pkl</code> / <code>.onnx</code> .	Backend en Node.js + Gemini AI: El servidor principal es Express. <i>Estrategia:</i> Crear un microservicio ligero o módulo JS/ONNX para cargar los modelos serializados requeridos y exponer los endpoints REST en Node.
14. Real-time Socket.IO	Pide visualización en tiempo real de inferencias y métricas en dashboard.	WebSockets Activos: Socket.IO ya se utiliza para eventos de usuario y sincronización Pro. <i>Estrategia:</i> Emitir canal de eventos de ML (<code>new_inference</code> , <code>model_alert</code>) al dashboard.
7. Estructura de Repositorio	Sugiere carpetas monolíticas raíz: <code>/api</code> , <code>/data</code> , <code>/notebooks</code> , <code>/models</code> .	Estructura Modular Activa: <code>/backend</code> , <code>/frontend</code> , <code>/smartwatch</code> , <code>/dashboard</code> , <code>/database</code> . <i>Estrategia:</i> Incorporar <code>/notebooks</code> y <code>/models</code> dentro del proyecto manteniendo la estructura de despliegue.

Justificación Técnica para la Evaluación

Es clave presentar ante los evaluadores que QUANTIFY supera los requerimientos mínimos al estar en producción con usuarios reales, justificando que los artefactos académicos (notebooks, scripts de simulación y modelos `.pkl`) complementan el ecosistema de producción sin comprometer su estabilidad.

2. Inventario de Entregables Faltantes

A partir de la revisión sistemática de las 21 etapas de la rúbrica, se han clasificado todas las actividades requeridas según su estado actual en el repositorio:

Etapa	Nombre de la Etapa	Estado	Entregable / Archivo Faltante
Etapa 1	Contexto y Problema	COMPLETO	Documentado en README.md y docs/etapa1/contexto.md.
Etapa 2	Metodología CRISP-DM	COMPLETO	Documentado en docs/etapa1/metodologia-datos.md y docs/gestion-pmbok.md.
Etapa 3	20 Propuestas de ML	FALTANTE	Crear docs/etapa1/proposals.md con 10 propuestas supervisadas y 10 no supervisadas.
Etapa 4	Fuentes, Entidades y Atributos	COMPLETO	Documentado en docs/etapa1/modelo-datos.md y esquemas de BD.
Etapa 5	Simulación del Dataset	FALTANTE	Script generador sintético reproducible con semilla 2026 y docs/simulation-rules.md.
Etapa 6	Modelado Data Warehouse	PARCIAL	Script DDL del Data Mart / Hechos y Dimensiones en database/warehouse/.
Etapa 7	Proceso ETL	FALTANTE	Scripts de extracción, limpieza y carga + reporte de calidad de datos.
Etapa 8	Análisis Exploratorio (EDA)	FALTANTE	Notebooks Jupyter en notebooks/eda/ con análisis univariado/bivariado/multivariado.
Etapa 9	Preparación de Datasets	FALTANTE	Script de particionado en carpetas: training, validation, test e inference.
Etapa 10	Modelos Supervisados	FALTANTE	Modelo entrenado de Clasificación/Regresión serializado en models/supervised/.
Etapa 11	Modelos No Supervisados	FALTANTE	Modelo de Clustering (K-Means/PCA) serializado en models/unsupervised/.
Etapa 12	Evaluación y Selección	FALTANTE	Documento docs/model-evaluation.md con tabla comparativa de métricas.
Etapa 13	Endpoints Inteligentes	PARCIAL	Rutas Express de inferencia ML + Documentación Swagger/OpenAPI (docs/api-contracts.md).

Etapas	Nombre de la Etapa	Estado	Entregable / Archivo Faltante
Etapa 14	Dashboard Socket.IO	PARCIAL	Conectar eventos de Socket.IO para emisión de inferencias y alertas de ML en tiempo real.
Etapa 15	Consumo Web / Wearable	COMPLETO	Frontend React y App Smartwatch integrados con el Backend.
Etapa 16	Pruebas Integrales	FALTANTE	Plan y suites de pruebas en tests/ (Dataset, ETL, Modelos, API y Sockets).
Etapa 17	Arquitectura de Software	COMPLETO	Diagramas e informe en docs/arquitectura.md.
Etapa 18	Ética y Privacidad	FALTANTE	Crear docs/ethics.md (Anonimización, datos sensibles de salud y sesgos).
Etapa 19	Uso Responsable de IA	FALTANTE	Crear docs/ai-usage.md (Prompts, herramientas de IA, validación y decisiones).
Etapa 20	Desarrollo Git/GitHub	COMPLETO	Estrategia de ramas (main, develop, feature/*) y commits semánticos activos.
Etapa 21	Presentación y Defensa	COMPLETO	Presentación PowerPoint y Guión en docs/presentation/.

3. Matriz de Asignación (5 Desarrolladores)

Para completar el 100% de la rúbrica integradora y preparar la entrega final, las actividades faltantes se distribuyen entre los 5 desarrolladores del equipo atendiendo a su especialidad técnica:

DEV 1

Tech Lead & Backend API

Desarrollador: Angel de Jesús (Líder)

- **Endpoints Inteligentes ML (Etapa 13):** Implementar rutas Express `/api/ml/supervised/predict` y `/api/ml/unsupervised/analyze`.
- **Contratos de API (Etapa 13):** Elaborar la documentación OpenAPI / Swagger en `docs/api-contracts.md`.
- **Coordinación General (Etapa 20):** Supervisión de Pull Requests en GitHub y matriz de trazabilidad con la rúbrica.

DEV 2

Data Science & ML Engineer

Desarrollador: Francisco García

- **Simulación de Dataset (Etapa 5):** Desarrollar script Python con `np.random.seed(2026)` y `docs/simulation-rules.md`.
- **Modelos Supervisados (Etapa 10):** Entrenar y serializar modelo de Clasificación de Riesgo de Estrés en `models/supervised/`.
- **Modelos No Supervisados (Etapa 11):** Entrenar y serializar Clustering K-Means de Hábitos en `models/unsupervised/`.

DEV 3

Data Analytics & ETL

Desarrollador: Jesús Alejandro Artiga

- **Análisis Exploratorio EDA (Etapa 8):** Crear notebooks Jupyter en `notebooks/eda/` con análisis univariado/bivariado/multivariado.
- **Pipeline ETL & Calidad (Etapa 7):** Desarrollar scripts de transformación y limpieza + reporte de calidad de datos.
- **Data Warehouse (Etapa 6):** Crear script DDL del Data Mart analítico (Hechos y Dimensiones) en `database/warehouse/`.

DEV 4

Frontend & Real-Time Sockets

Desarrollador: AI Farías Leyva

- **Dashboard Socket.IO (Etapa 14):** Conectar eventos WebSockets de inferencias (`new_inference`, `model_alert`) a la UI.
- **Consumo en Frontend & Wearable (Etapa 15):** Integrar componentes React para consultar endpoints inteligentes de ML.
- **Storytelling Visual (Etapa 14):** Diseñar widgets interactivos de predicción, clusters y nivel de confianza en el Dashboard.

DEV 5

QA, Ética & Documentación Técnica

Desarrollador: 5º Integrante del Equipo

- **20 Propuestas de ML (Etapa 3):** Redactar `docs/etapa1/proposals.md` con la tabla de 10 propuestas supervisadas y 10 no supervisadas.
- **Uso Responsable de IA (Etapa 19):** Elaborar el documento obligatorio `docs/ai-usage.md` (prompts, código generado y validaciones).
- **Ética y Privacidad (Etapa 18):** Elaborar `docs/ethics.md` (anonimización de datos de salud y mitigación de sesgos).
- **Plan de Pruebas Integrales (Etapa 16):** Diseñar y ejecutar suites de pruebas para el dataset, ETL, modelos y API en `tests/`.

4. Cronograma de Ejecución (Roadmap)

Para asegurar la entrega oportuna de todos los componentes requeridos por la rúbrica integradora, se establece el siguiente plan de trabajo organizado en 3 iteraciones (Sprints):

Sprint / Período	Entregables y Objetivos Clave	Responsables
Sprint 1 (Días 1 - 4) <i>Fase Datos y Documentación</i>	- Redacción de docs/proposals.md (20 propuestas ML). - Script generador sintético con np.random.seed(2026). - Creación de Notebooks EDA en notebooks/eda/. - Elaboración de docs/ethics.md y docs/ai-usage.md.	DEV 2 (Data Science) DEV 3 (Analytics & ETL) DEV 5 (QA & Docs)
Sprint 2 (Días 5 - 8) <i>Fase Modelos y API</i>	- Entrenamiento y serialización de Modelos ML (models/). - Creación de script ETL y particionado de datos (data/). - Implementación de Endpoints inteligentes Express. - Generación de contratos OpenAPI / Swagger (docs/api-contracts.md).	DEV 1 (Tech Lead / API) DEV 2 (Data Science) DEV 3 (Analytics & ETL)
Sprint 3 (Días 9 - 12) <i>Fase Frontend, Sockets & QA</i>	- Integración de eventos Socket.IO para inferencias en tiempo real. - Consumo de endpoints de ML desde React Dashboard y Smartwatch. - Ejecución del Plan de Pruebas integrales (tests/). - Ensamble final de la documentación y defensa del proyecto.	DEV 1 (Tech Lead) DEV 4 (Frontend & Sockets) DEV 5 (QA & Pruebas)

Resultado Esperado

Con este plan de asignación, el equipo garantizará el **100% de cumplimiento de la rúbrica del Proyecto Integrador**, respaldado por la solidez de una aplicación real en producción que ya cumple con altos estándares de ingeniería.